NLP COURSE PROJECT · TECHNICAL REPORT

Building the subtitle corrector's models from scratch

How the five self-trained models of the audio-aware subtitle correction system were designed, implemented, trained and evaluated: architectures, layers, parameters, code, data, results, and the problems solved along the way.

Shane · B.Tech CSE (AI & Data Analytics), Chennai · 19 October 2026

5 models designed, coded and trained from scratch	**30.0M** trainable neural parameters (3 PyTorch networks)	**13.4M** n-gram entries in our Kneser-Ney language model
1.16B pretrained parameters replaced	**8.54M** words of training text (LibriSpeech-PC train)	**100.6 h** of training speech (28,538 utterances, 251 speakers)

The one-sentence version

Except for Whisper, which supplies the audio evidence, every learned component of the system (the language model, the punctuation tagger, the grammar and ASR-error corrector, the part-of-speech tagger and the forced aligner) was written from first principles in NumPy and PyTorch and trained on public data on one 6 GB laptop GPU.

Contents

1. Executive summary . . . 3

2. The system and why we replaced its models . . . 4

3. Datasets and how we used them . . . 6

4. Model 1: Kneser-Ney n-gram language model . . . 7

5. Model 2: BiLSTM punctuation tagger . . . 9

6. Model 3: Transformer corrector with our own BPE . . . 11

7. Model 4: averaged-perceptron POS tagger . . . 14

8. Model 5: CTC acoustic model for forced alignment . . . 16

9. Putting the models to work . . . 20

10. End-to-end results: before and after . . . 21

11. Difficulties faced and how we solved them . . . 23

12. Lessons learned and future work . . . 25

13. Presentation kit . . . 26

14. Numbers cheat sheet . . . 28

15. Appendix . . . 29

AT A GLANCE

1. Executive summary

The project corrects subtitles using the audio as evidence. Mode A takes a video and its faulty *.srt/.vtt*; Mode B generates subtitles from audio alone. Both modes produce corrected text, word-accurate timing and subtitle cues that follow Netflix/BBC reading-speed and line-length rules. The first version relied on six large pretrained models. This report describes how five of them were replaced with models built and trained from scratch.

Model	What we built	Size	Headline result
Fusion language model	Interpolated modified Kneser-Ney 4-gram, open vocabulary, NumPy	13.4M n-grams	dev perplexity 284; trained in 114 s
Punctuation	Word + char-CNN + casing embeddings, 2-layer BiLSTM	12.26M	macro-F1 0.810 (full stop 0.913)
Corrector	Transformer encoder-decoder (4+3 layers) + own BPE	8.41M	noisy-dev WER 13.87% -> 4.26%
POS tagger	Averaged perceptron, 17 feature templates, UD tags	127,047 features	held-out accuracy 97.21%
Forced aligner	Log-mel + CNN + 4-layer BiLSTM, CTC, Viterbi alignment	9.32M	mean onset error 25 ms; dev CER 7.8%

What makes this work stand out

- **39 times fewer parameters.** The neural replacements total 30.0M parameters, against about 1.16 billion in the pretrained models they replace (XLM-R-large alone has 560M).
- **Everything is our own code.** The n-gram counting, Kneser-Ney smoothing, BPE tokenizer, mel filterbank, CTC Viterbi alignment, beam search and perceptron are implemented in this repository, not imported.
- **Trained on a laptop.** All five models train on one RTX 4050 (6 GB) and a 16 GB laptop; the two count-based models each train in under four minutes on the CPU.
- **Honest evaluation.** Strict train/dev/test separation, model selection on dev only, and the same benchmark as the pretrained version, so the before/after comparison is like for like.
- **No loss of accuracy.** On the same 27-file test set, WER went from 3.03% to 2.75% and the mean cue-onset error from 50 to 39 ms after the swap; only punctuation F1 is lower (72.6 to 70.1). Section 10 explains what caused each change.
- **Engineered to survive failure.** Every neural trainer resumes from checkpoints, which mattered: the network dropped, a 6.4 GB download arrived corrupted, and the agent session lost its connection mid-training.

> **End-to-end result**
> On the 27-file test set (Mode A), WER is 2.75% with our models vs 3.03% with the pretrained ones; ERRANT F0.5 77.1 vs 75.1; punctuation F1 70.1 vs 72.6; mean cue-onset error 39 ms vs 50 ms. Section 10 has the full comparison.

BACKGROUND

2. The system and why we replaced its models

The pipeline separates *what was said* (speech recognition plus evidence fusion) from *when it was said* (forced alignment). Figure 1 shows where each self-trained model sits. Whisper is the only pretrained network left; everything shaded blue was built in this project.

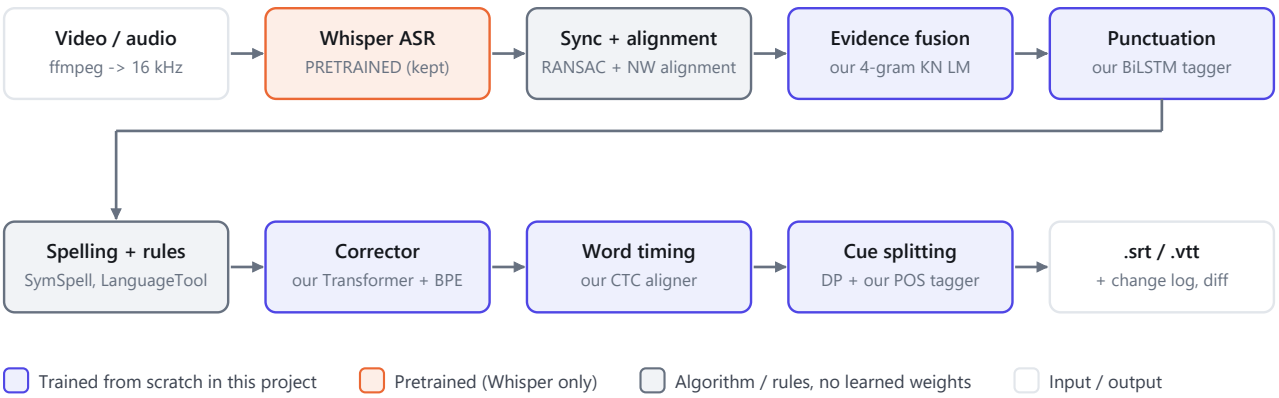


Figure 1. Processing pipeline. Mode A runs every box; Mode B skips sync and fusion and uses the Whisper transcript as the draft.

Why build our own models?

- **Learning.** Using a pretrained checkpoint hides the interesting part. Building the models makes the course concepts concrete: smoothing, sequence labelling, subword tokenization, attention, CTC and structured prediction.
- **Size and speed.** The old text models needed about 860M parameters on a 6 GB GPU; ours need about 21M.
- **Control over data.** Every model is trained on known, public, licensed data with a documented split, so there is no risk that a pretrained model has already seen the test set.
- **Licences.** The MMS aligner weights are CC-BY-NC and vennify T5 is CC-BY-NC-SA. Our models carry no such restrictions beyond those of the datasets.
- **Transparency.** Every decision can be inspected: n-gram counts, perceptron features, the BPE merge list, attention masks, CTC alignment paths.

What we kept, and why

- **Whisper (speech recognition).** Whisper was trained on more than 680,000 hours of speech. We estimate that a recogniser trained on our 100 hours would reach roughly 15-30% WER instead of about 3%, and the audio evidence the whole system depends on would collapse. We treat Whisper as a sensor, the way a vision project treats a camera.
- **Evaluation instruments only.** The Montreal Forced Aligner builds the silver timing references, whisper-tiny produces realistic ASR errors for the synthetic benchmark, and ERRANT uses spaCy to classify edits. They measure the system; they are not part of it.

Replacement map

Job	Before (pretrained)	Params	Now (ours, from scratch)	Size
Fusion language model	distilgpt2	82M	Kneser-Ney 4-gram (NumPy)	13.4M n-grams (no weights)
Punctuation restoration	FullStop XLM-RoBERTa-large	560M	BiLSTM tagger + char-CNN	12.26M
Grammar / ASR-error corrector	vennify T5-base (+ CoEdIT-large 770M)	220M	Transformer enc-dec + BPE	8.41M
Part-of-speech tags for line breaks	spaCy en_core_web_sm (12 MB CNN pipeline)	~12 MB	Averaged perceptron	127,047 sparse features
Forced alignment (word timing)	MMS-300M wav2vec2 aligner	300M	CNN + BiLSTM CTC model	9.32M

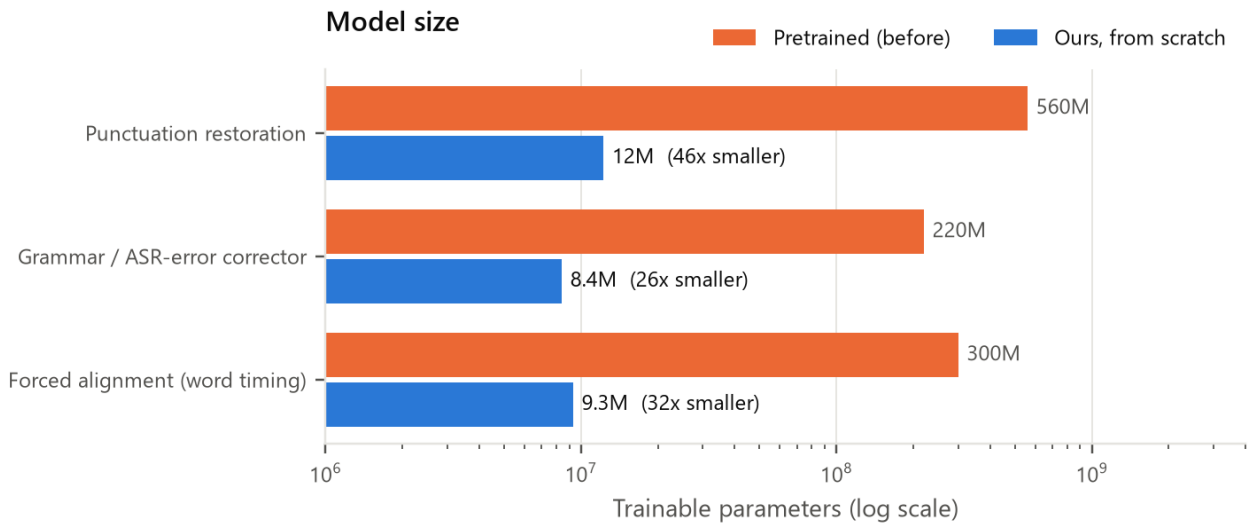


Figure 2. Parameter counts on a log scale. The language model and POS tagger are count-based and have no dense weights, so they are not plotted.

DATA

3. Datasets and how we used them

All training data is public. Only **train** splits are used for learning; **dev** splits choose hyper-parameters and checkpoints; the **test** benchmark is touched only for the final evaluation.

Dataset	Content	Size used	Used for
LibriSpeech-PC train text	Punctuated, cased transcripts of LibriSpeech train-clean-100/360 + train-other-500 (Meister et al. 2023)	563,049 sentences · 8,544,054 words	LM, punctuation, corrector, BPE
LibriSpeech train-clean-100 audio	Read English speech, 16 kHz (Panayotov et al. 2015)	28,538 utterances · 100.6 h · 251 speakers	CTC acoustic model
Brown corpus	1.16M tokens of 1961 American English, 87 tags (Francis & Kucera)	54,473 train / 2,867 held-out sentences	POS tagger
whisper-tiny confusions	Real ASR errors of a weak recogniser on the dev clips (RoBART idea)	268 word -> mistake entries	Corrector noise
LibriSpeech-PC dev-clean/other	Dev transcripts	98,053 words	LM perplexity, punctuation and corrector selection
dev-clean audio + MFA references	Silver word timings from the Montreal Forced Aligner	300 utterances + 4 long-form clips	Aligner selection
Benchmark (test)	8 LibriSpeech test-clean + 8 test-other clips (3-5 min) and 11 TED-LIUM talks	27 files, seeded corruptions	Final evaluation only

Keeping the evaluation honest

- LibriSpeech train, dev and test speakers are disjoint, so the acoustic model is always evaluated on unseen voices.
- The corrector's real ASR confusions come from the *dev* clips; the test clips never feed any training step.
- Fusion weights and pipeline settings were re-tuned on dev only; test numbers were produced once, at the end.
- The synthetic corruption generator is seeded, and every injected error is logged, so results are reproducible.

A closer look at the punctuation labels

Each training word carries one label. The classes are very unbalanced, which is why we report macro-F1 rather than accuracy: a model that never predicts punctuation would already be 87.3% accurate.

Label	Meaning	Training words	Share
0	no mark after the word	7,460,984	87.3%
.	full stop (also ! and sentence ends)	382,057	4.5%
,	comma (also ; and :)	665,591	7.8%
?	question mark	35,422	0.41%

LANGUAGE MODELLING

4. Model 1: Kneser-Ney n-gram language model

Job. When the subtitle says *s* but Whisper heard *a*, fusion asks which word fits the context better. The language model supplies log P(word | previous words); the difference log P(a | ctx) - log P(s | ctx) is one of the fusion features. It replaces distilgpt2 (82M parameters).

The mathematics

We implemented **interpolated modified Kneser-Ney** smoothing (Chen & Goodman 1998), the strongest classical n-gram method. For an n-gram with history h and count c:

$$P_k(w \mid h) = \max(c_k(hw) - D_k(c), 0) / \Sigma_v\, c_k(hv) \;+\; \gamma_k(h) \cdot P_{k-1}(w \mid h')$$
$$\gamma_k(h) = [\, D_1 \cdot N_1(h\cdot) + D_2 \cdot N_2(h\cdot) + D_{3+} \cdot N_{3+}(h\cdot)\,] / \Sigma_v\, c_k(hv)$$
$$Y = n_1 / (n_1 + 2n_2), \quad D_1 = 1 - 2Y \cdot n_2/n_1, \quad D_2 = 2 - 3Y \cdot n_3/n_2, \quad D_{3+} = 3 - 4Y \cdot n_4/n_3$$

Lower orders do not use raw counts but **continuation counts** $N_{1+}(\cdot w)$, the number of different words that precede w. This is Kneser-Ney's key idea: "Francisco" is frequent but almost always follows "San", so it should get a small unigram probability. $n_1..n_4$ are the count-of-counts of each order, and the recursion ends in a uniform distribution.

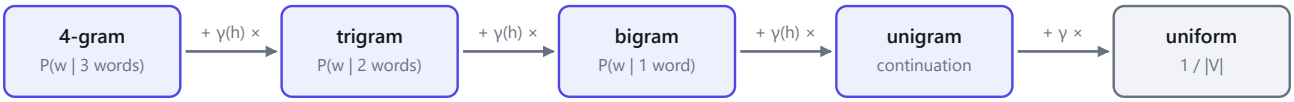


Figure 3. Interpolation chain: each order discounts its counts and passes the saved mass down.

Engineering it to fit a laptop

- **Bit-packed n-grams.** The vocabulary has 55,757 words, which fits in 16 bits, so a whole 4-gram packs into one 64-bit integer. Counting is then a single `np.unique` over the 8,544,054-token stream, and lookup is a binary search (`np.searchsorted`). No Python dictionaries hold n-grams.
- **Streaming corpus.** Sentences come from a generator read twice (vocabulary pass, counting pass), so the corpus is never held in memory as Python strings. Training took **114 s** on the CPU.
- **Open vocabulary.** 29,293 word types seen only once become <unk>. The probability of an unknown word is P(<unk> | h) times a character-bigram spelling model trained on those rare words, so a typo like "recieve" scores far lower than "receive" in the same context.
- **Vectorised batch scoring.** Fusion scores thousands of short candidate windows at once; all orders are evaluated with array operations.

> **Talking point**
> Our count-based LM has no neural weights and trains in under two minutes. After re-tuning on dev, fusion with it reaches F0.5 0.747, against 0.740 with the 82M-parameter distilgpt2. To be fair, the small gain comes from searching a wider range for the LM's insertion gate, which the n-gram scores needed, not from the LM being better than a neural one.

src/subfix/ngram.py (line 188) · batched interpolated lookup, highest order last

```
p = np.full(n, 1.0 / (len(self.vocab) - 1))          # P_0: uniform (excluding <s>)
for k in range(1, self.order + 1):
    keys, probs = self.t[f"k{k}"], self.t[f"p{k}"]
    hkeys, gam = self.t[f"h{k}"], self.t[f"g{k}"]
    if k == 1:
        avail = np.ones(n, dtype=bool)
        hkey = np.zeros(n, dtype=np.uint64)
    else:
        h = hist[:, self.order - k:]                 # the k-1 most recent words
        avail = np.all(h >= 0, axis=1)
        hkey = np.zeros(n, dtype=np.uint64)
        for j in range(k - 1):
            hkey = (hkey << B) | np.where(avail, h[:, j], 0).astype(np.uint64)
    hi = np.searchsorted(hkeys, hkey)
    hi_c = np.minimum(hi, len(hkeys) - 1)
    seen = avail & (hkeys[hi_c] == hkey)
    g = np.where(seen, gam[hi_c], 1.0)
    key = (hkey << B) | w
    ki = np.minimum(np.searchsorted(keys, key), len(keys) - 1)
    hit = seen & (keys[ki] == key)
    top = np.where(hit, probs[ki], 0.0)
    p = np.where(avail, top + g * p, p)
```

Model statistics

Order	n-grams stored	Histories	Discounts D1 / D2 / D3+
1	55,756	1	0.146 / 1.746 / 2.618
2	1,678,610	55,756	0.738 / 1.073 / 1.379
3	4,860,231	1,644,956	0.849 / 1.152 / 1.355
4	6,799,117	4,613,916	0.914 / 1.288 / 1.451

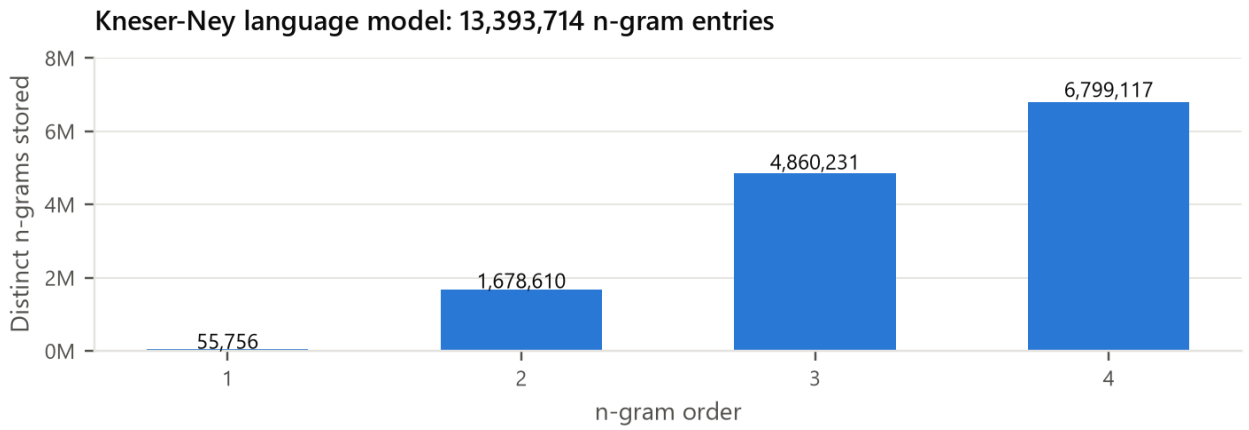


Figure 4. Number of distinct n-grams per order.

55,757
vocabulary (words)

284
perplexity on dev-clean

252
perplexity on dev-other

0.90%
out-of-vocabulary rate (dev-clean)

114 s
training time (CPU)

226 MB
model file

SEQUENCE LABELLING

5. Model 2: BiLSTM punctuation tagger

Job. Speech recognition and many subtitles lack punctuation. We treat restoration as **sequence labelling**: every word gets one of four labels (none, full stop, comma, question mark) that says which mark follows it. It replaces FullStop XLM-RoBERTa-large (560M parameters).

Architecture

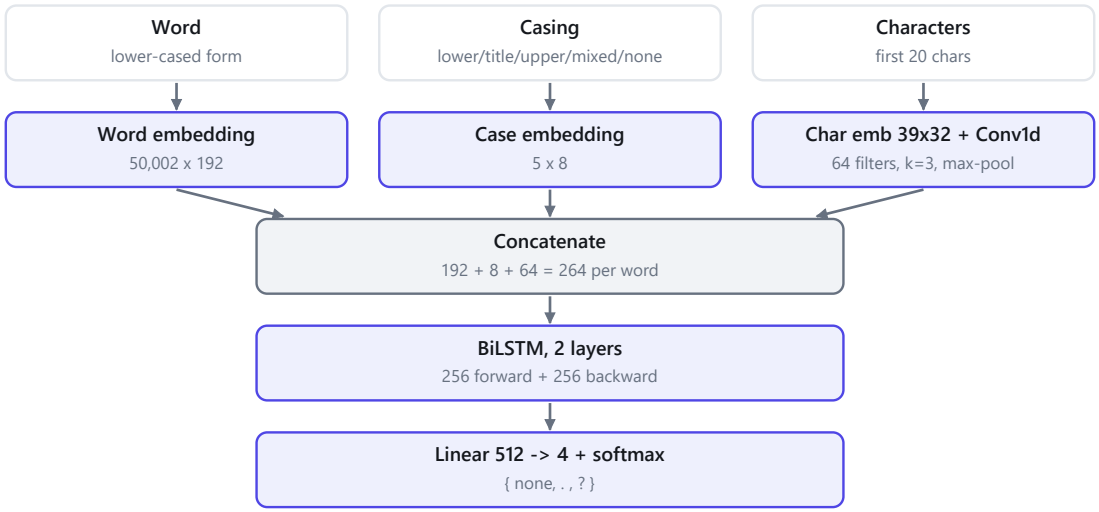


Figure 5. Each word is represented three ways, and a bidirectional LSTM reads the whole window so that the label can depend on words on both sides.

Layer	Type	Parameters
word	Embedding	9,600,384
case	Embedding	40
char	Embedding	1,248
conv	Conv1d	6,208
lstm	LSTM	2,646,016
out	Linear	2,052
Total		**12,255,948**

- **Why a character CNN?** Subtitles contain typos and rare names. The CNN builds a representation from spelling, so even an unseen word gets a useful vector.
- **Why a casing embedding?** A capitalised word often starts a sentence, which signals a full stop before it. Half of the training windows are lower-cased at random, so the model also works on caseless subtitles.
- **Why bidirectional?** Whether a comma follows a word depends on what comes next ("..., which", "..., and"), so the right-hand context matters as much as the left.

src/subfix/textfix/punct.py (line 67) · the forward pass

```
def forward(self, words, cases, chars):
    # words, cases: (B, L); chars: (B, L, C)
    B, L, C = chars.shape
    ce = self.char(chars.view(B * L, C)).transpose(1, 2)            # (B*L, char_dim, C)
    ce = torch.relu(self.conv(ce)).masked_fill((chars.view(B * L, 1, C) == 0), 0.0)
    ce = ce.max(dim=2).values.view(B, L, -1)
    x = torch.cat([self.word(words), self.case(cases), ce], dim=-1)
    h, _ = self.lstm(self.drop(x))
    return self.out(self.drop(h))
```

Training

Setting	Value
Data	8,544,054 words from whole LibriSpeech chapters (utterances in order)
Windows	200 words, batch 32, new random offsets every epoch
Optimiser	AdamW, lr 0.002, weight decay 0.01, one-cycle schedule (5% warm-up), gradient clip 1.0
Regularisation	dropout 0.3 on inputs and outputs; 50% of windows lower-cased (augmentation)
Epochs / time	8 epochs of about 4.5 min on the RTX 4050, plus 2.5 min of preprocessing: about 41 min in total
Selection	best dev score = mean of cased and lower-cased macro-F1
Inference	overlapping 230-word windows; each word keeps the prediction made farthest from a window edge

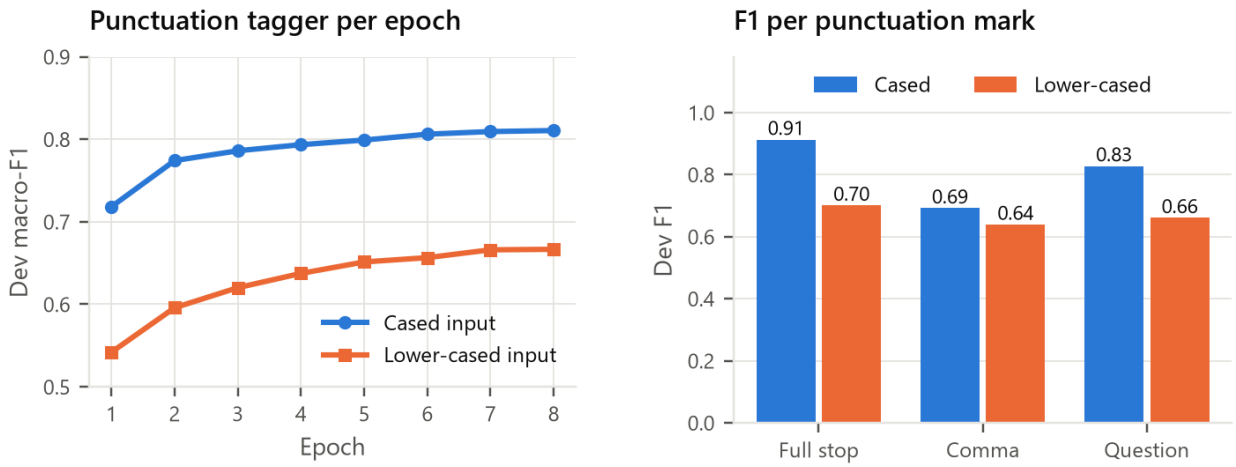


Figure 6. Left: dev macro-F1 per epoch. Right: F1 per mark for the best checkpoint, with and without casing in the input.

Talking point

A 12.3M-parameter BiLSTM trained in about 41 minutes reaches macro-F1 0.81 and places full stops with F1 0.91. It is 46 times smaller than the XLM-R model it replaces. Commas are the hardest class, which matches the literature, because comma use is partly a matter of style.

SEQUENCE-TO-SEQUENCE

6. Model 3: Transformer corrector with our own BPE

Job. Fix what rules cannot: real-word errors, dropped or duplicated function words, homophones and ASR-style confusions, sentence by sentence. It replaces vennify T5-base (220M) and the CoEdIT-large (770M) comparison model.

Step 1: a byte-pair-encoding tokenizer, written from scratch

BPE (Sennrich et al. 2016) starts from characters and repeatedly merges the most frequent adjacent pair. Ours keeps case and attached punctuation, marks word ends with </w> so decoding is exact, and learned **7,888 merges** over a 108-symbol alphabet, giving a **8,000-token vocabulary** (4 special tokens included). Incremental pair counts and a lazy max-heap make training take 87 s in pure Python on the CPU.

Input sentence	Our BPE pieces
The recieved subtitles were uncorrected.	The_ | re | ci | eved_ | sub | tit | les_ | were_ | un | correc | ted._

Frequent words stay whole ("were"); a misspelling such as "recieved" splits into familiar pieces, so the model can still represent it.

Step 2: the Transformer

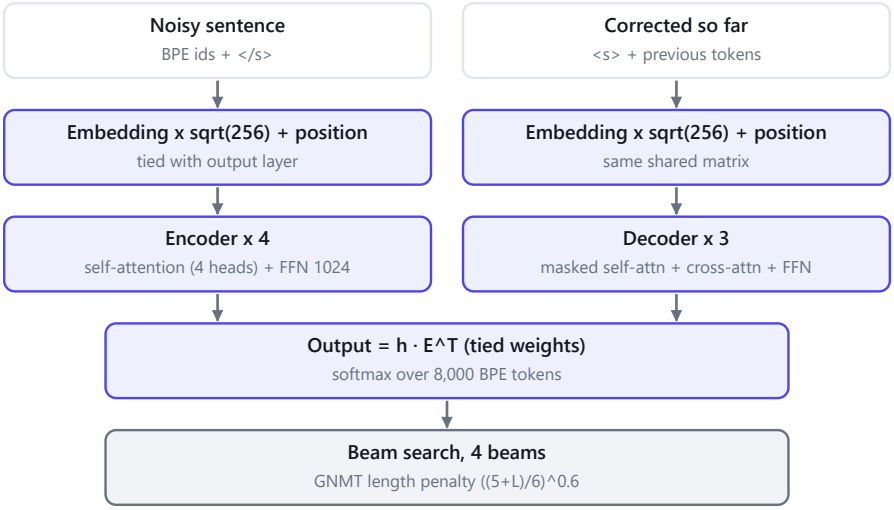


Figure 7. A pre-norm Transformer encoder-decoder. The embedding matrix is shared between source, target and output projection, which saves 2M parameters and helps with only 8.4M in total.

Component	Shape	Parameters
Token embedding (shared, tied)	8,000 x 256	2,048,000
Learned positions	160 x 256	40,960
Encoder (4 layers + final LayerNorm)	789,760 per layer	3,159,552
Decoder (3 layers + final LayerNorm)	1,053,440 per layer	3,160,832
Total	**d_model 256, 4 heads, FFN 1024, dropout 0.1**	**8,409,344**

`src/subfix/textfix/seq2seq.py` (line 36) · embedding, encoder, decoder with tied output

```
def embed(self, ids):
    pos = torch.arange(ids.shape[1], device=ids.device).unsqueeze(0)
    return self.drop(self.emb(ids) * self.scale + self.pos(pos))

def encode(self, src):
    mask = src == PAD
    return self.encoder(self.embed(src), src_key_padding_mask=mask), mask

def decode(self, tgt, memory, mem_mask):
    T = tgt.shape[1]
    causal = torch.triu(torch.ones((T, T), dtype=torch.bool, device=tgt.device), diagonal=1)
    h = self.decoder(self.embed(tgt), memory, tgt_mask=causal, tgt_is_causal=True,
                     tgt_key_padding_mask=tgt == PAD, memory_key_padding_mask=mem_mask)
    return h @ self.emb.weight.t()

def forward(self, src, tgt_in):
    memory, mem_mask = self.encode(src)
    return self.decode(tgt_in, memory, mem_mask)
```

Step 3: training data without a parallel corpus

No dataset of (faulty subtitle, corrected subtitle) pairs exists, so we **manufacture** them. Each of the 513,355 clean training sentences is corrupted on the fly with the same operations as the benchmark, plus real whisper-tiny mistakes. A third of the pairs stay clean so the model learns that copying correct text is usually the right answer. Fresh noise is drawn every epoch, so the model sees 15 different noisy versions of each sentence.

Noise	Rate (up to)	Example (illustrative)
Keyboard typo (insert / delete / swap / substitute)	4% of words	receive -> recieve
Homophone swap	3%	their -> there
Real whisper-tiny confusion (from dev clips)	4%	a word whisper-tiny misheard
Dropped function word	2%	went to the -> went the
Duplicated word	1%	the the
Punctuation: strip all / drop half	30% / 30% of sentences	Hello, world. -> Hello world
Lower-casing	30% of sentences	Paris -> paris
Left completely clean	33% of pairs	(copy task)

`experiments/train_corrector.py` (line 62) · the noise generator (word-level part)

```
def noisify(sent: str, rng: random.Random, conf: dict[str, list[str]]) -> str:
    from corrupt import typo
    hom = T.homophone_groups()
    func = T.function_words()
    out = []
    for tok in sent.split():
        pre, core, suf = T.split_affixes(tok)
        r = rng.random()
        if r < 0.04 and sum(ch.isalpha() for ch in core) >= 3:
            out.append(pre + typo(core, rng) + suf)
        elif r < 0.07 and core.lower() in hom:
            new = rng.choice(hom[core.lower()])
            out.append(pre + (new.capitalize() if core[:1].isupper() else new) + suf)
        elif r < 0.11 and T.canon(tok) in conf:
            new = rng.choice(conf[T.canon(tok)])
            out.append(pre + (new.capitalize() if core[:1].isupper() else new) + suf)
        elif r < 0.13 and T.norm(tok) in func and not suf:
            continue
        elif r < 0.14:
            out.extend([pre + core, tok])
        else:
            out.append(tok)
```

Training setup and results

Setting	Value
Batching	length-bucketed, at most 6,000 tokens per batch (2,297 batches per epoch)
Optimiser	AdamW (betas 0.9/0.98, weight decay 0.01), peak lr 0.0007, 2,000 warm-up steps, cosine decay to 5%
Loss	cross-entropy with label smoothing 0.1, padding ignored
Precision	bfloat16 autocast on the GPU (same range as fp32, so no overflow), gradient clip 1.0
Length	15 epochs = 34,486 steps, about 76 min in total
Selection	lowest WER on 1,000 noisy dev-clean sentences; 500 clean ones check over-correction

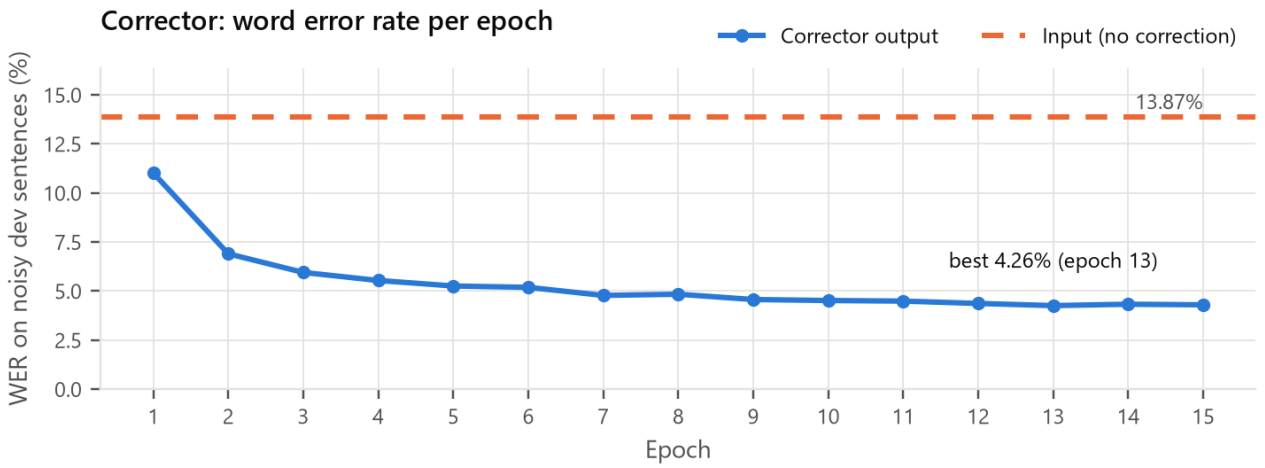


Figure 8. Word error rate on noisy dev sentences. The dashed line is the error of the uncorrected input.

13.87%
WER of the noisy input

4.26%
WER after correction

69%
relative error reduction

32.5%
sentences restored exactly

97.6%
clean sentences left untouched

8.4M
parameters (26x smaller than T5-base)

Safety rails around the corrector

- **Edit-ratio guard:** an output that changes more than 15% of a sentence's words (tuned on dev) is rejected.
- **Audio-consistency guard:** the corrector may not replace or delete a word Whisper heard with probability 0.8 or more; the audio has the final say.
- **Copy training:** a third of the training pairs are clean, which is why 97.6% of clean sentences come out unchanged.

STRUCTURED PREDICTION

7. Model 4: averaged-perceptron POS tagger

Job. The cue segmenter must not break a line between "the" and its noun, or end a line with a preposition. It needs a part-of-speech tag for every word. This replaces spaCy's en_core_web_sm.

Algorithm

- **Greedy left-to-right tagging** with a linear model: score(tag) = Σ w[feature, tag], take the arg-max, and feed the predicted tag into the next word's features.
- **Perceptron learning** (Collins 2002): when the guess is wrong, add 1 to the weights of the true tag's features and subtract 1 from the guessed tag's.
- **Averaging** keeps the mean of every weight over all updates, which makes the perceptron far more stable. We use timestamps, so averaging costs nothing per step.
- **Tag dictionary:** 3,808 frequent, unambiguous words (at least 20 occurrences, at least 97% one tag) skip the model entirely, which is both faster and more accurate.

Feature templates (17)	
bias; current word; suffixes (3, 2 chars); first letter	word identity and morphology
shape (UPPER / Title / digit / punct / lower); shape + sentence-initial	capitalisation cues
previous tag; tag two back; both together; previous tag + current word	tag context (a Markov chain)
previous word and its suffix; word two back	left context
next word and its suffix; word two ahead	right context

`src/subfix/postag.py` (line 189) · the averaged-perceptron update

```
if not guess:
    feats = self._features(i + 2, word, context, words, prev, prev2)
    guess = self._predict(feats)
    n_updates += 1
    if guess != gold:
        for f in feats:
            wts = self.weights.setdefault(f, {})
            for c, delta in ((gold, 1.0), (guess, -1.0)):
                key = (f, c)
                totals[key] += (n_updates - stamps[key]) * wts.get(c, 0.0)
                stamps[key] = n_updates
                wts[c] = wts.get(c, 0.0) + delta
prev2, prev = prev, guess
correct += guess == gold
```

Data: from Brown tags to Universal Dependencies

The Brown corpus uses 87 fine-grained tags plus compounds such as `PPS+BEZ` ("it's") and `NN$-TL`. Our segmenter's rules use the 17 Universal POS tags, so we wrote a mapping with contextual rules: forms of *be* and modals become AUX, and *have/do* become AUX only when a verb follows within three words ("has gone" vs "has a car").

Results

97.21%
held-out accuracy (48,953 tokens)

127,047
active features after averaging

188 s
training time, 5 passes (CPU)

Pass	1	2	3	4	5
Training accuracy	95.08%	96.32%	97.12%	97.56%	97.88%

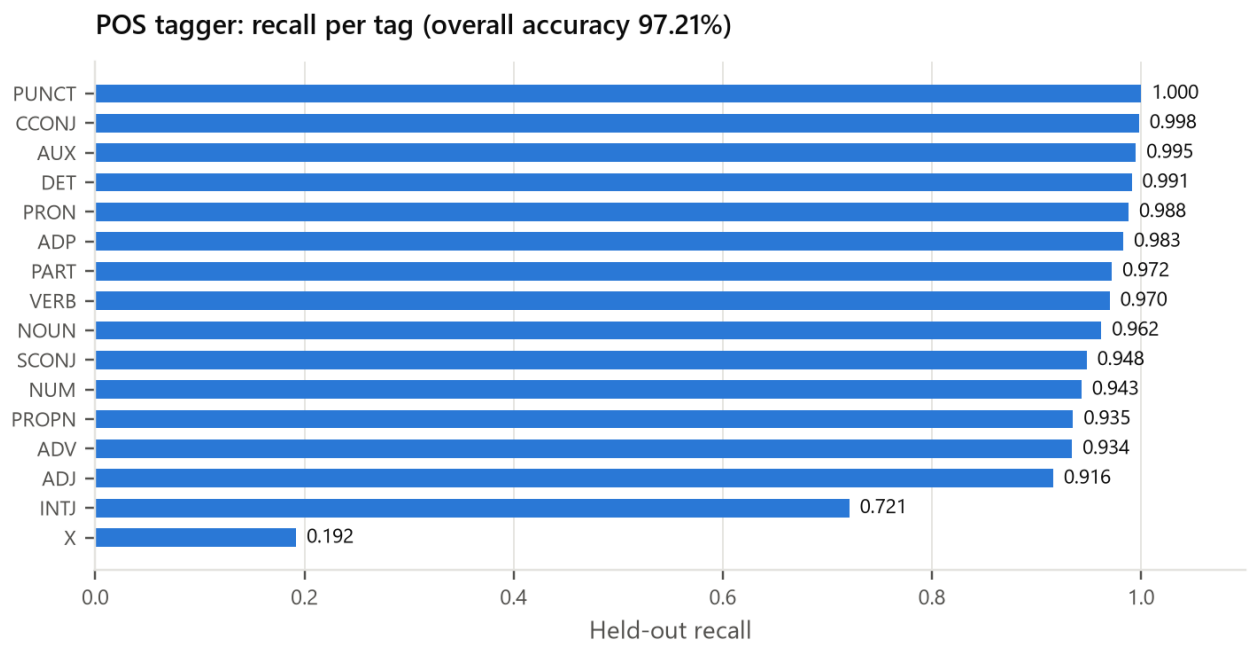


Figure 9. Recall per tag on held-out Brown sentences. X (foreign words) and INTJ are rare; the tags the segmenter relies on (DET, ADP, CCONJ, AUX) are above 98%.

Talking point

A model with no neural network at all, trained in about three minutes on a CPU, tags 97.2% of unseen words correctly, close to the roughly 97% of classic published taggers on comparable data.

SPEECH

8. Model 5: CTC acoustic model for forced alignment

Job. After the text is corrected, every word needs an exact start and end time so that cues appear when the words are spoken. We train a character-level acoustic model with **CTC** (Graves et al. 2006) and use it for **forced alignment**: the transcript is known, and we only search for *where* each character is. It replaces the 300M-parameter MMS wav2vec2 aligner.

Architecture

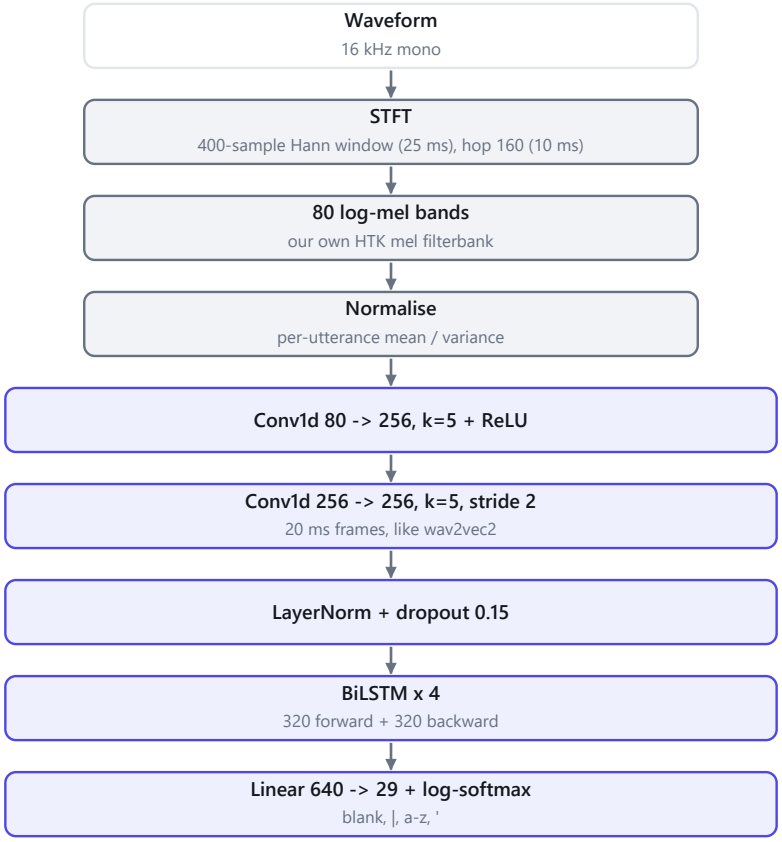


Figure 10. The acoustic model. The feature extractor is part of the model and runs on the GPU.

Layer	Type	Parameters
conv1	Conv1d	102,656
conv2	Conv1d	327,936
norm	LayerNorm	512
lstm	LSTM	8,867,840
out	Linear	18,589
Total		**9,317,533**

src/subfix/acoustic.py (line 71) · the log-mel front end, written from scratch

```
def forward(self, wav, lengths):
    spec = torch.stft(wav.float(), N_FFT, HOP, window=self.window, return_complex=True, center=True)
    power = spec.real ** 2 + spec.imag ** 2                    # (B, F, T)
    mel = torch.log(torch.clamp(self.fb @ power, min=1e-10)).transpose(1, 2)
    n = lengths // HOP + 1
    mask = (torch.arange(mel.shape[1], device=mel.device)[None, :] < n[:, None]).unsqueeze(-1)
    cnt = mask.sum(1, keepdim=True).clamp(min=1)
    mean = (mel * mask).sum(1, keepdim=True) / cnt
    var = (((mel - mean) * mask) ** 2).sum(1, keepdim=True) / cnt
    return ((mel - mean) / torch.sqrt(var + 1e-5)) * mask, n
```

How CTC works, in three lines

- The network outputs, every 20 ms, a probability for each of the 29 symbols, including a special **blank**.
- A transcript can be spelled across the frames in many ways (repeats collapse, blanks vanish); the **CTC loss** sums the probability of all of them with dynamic programming, so no frame-level labels are needed.
- For alignment we take the **single best path** (Viterbi) that spells exactly our transcript. The frames where a word's letters appear give its start and end.

src/subfix/timing.py (line 86) · our NumPy Viterbi over the CTC trellis

```
def ctc_viterbi(logp: np.ndarray, targets: list[int], blank: int) -> np.ndarray | None:
    """Best CTC path; returns the extended-state index per frame (odd = target token)."""
    T_, L = logp.shape[0], len(targets)
    if L == 0:
        return None
    S = 2 * L + 1
    ext = np.full(S, blank, dtype=np.int64)
    ext[1::2] = targets
    min_frames = L + sum(1 for k in range(1, L) if targets[k] == targets[k - 1])
    if T_ < min_frames:
        return None
    skip_ok = np.zeros(S, dtype=bool)
    skip_ok[3::2] = ext[3::2] != ext[1:-2:2]
    neg = -1e30
    alpha = np.full(S, neg)
    alpha[0] = logp[0, blank]
    alpha[1] = logp[0, ext[1]]
    bp = np.zeros((T_, S), dtype=np.int8)
    em = logp[:, ext]                          # (T, S)
    for t in range(1, T_):
        c1 = np.concatenate([[neg], alpha[:-1]])
        c2 = np.concatenate([[neg, neg], alpha[:-2]])
        c2 = np.where(skip_ok, c2, neg)
        stack = np.stack([alpha, c1, c2])
        choice = stack.argmax(axis=0)
        alpha = stack[choice, np.arange(S)] + em[t]
        bp[t] = choice
    s = S - 1 if alpha[S - 1] >= alpha[S - 2] else S - 2
    path = np.empty(T_, dtype=np.int64)
    for t in range(T_ - 1, -1, -1):
        path[t] = s
        s -= int(bp[t, s])
    return path
```

Training

Setting	Value
Data	LibriSpeech train-clean-100: 28,538 utterances, 100.6 h, 251 speakers (the whole split; utterances over 20 s are skipped)
Augmentation	SpecAugment: 2 frequency masks (up to 15 bands) and 2 time masks (up to 5%)
Batching	utterances sorted by length, at most 320 s of audio per batch, 2 loader workers
Optimiser	AdamW, peak lr 1.5e-3, 1,000 warm-up steps, cosine decay to 2%, gradient clip 5
Epochs / time	11 epochs (13,287 steps), 2 h 56 min wall clock, 8-21 min per epoch. Planned 15, cut to 11 because the onset error had been flat since epoch 1; the schedule was resumed with a shorter cosine decay
Precision	network in fp16 autocast with a gradient scaler; the STFT stays in fp32 (power overflows fp16)
Checkpoints	every 500 steps and every epoch; training resumes from the exact batch
Selection	mean word-onset error against MFA references on dev clips, plus greedy CER on 300 dev utterances
Pilot run	while the damaged archive was being repaired: 2 epochs on the first 40.6 h (11,353 utterances, 102 speakers) to confirm that CTC trained from scratch converges; not used for the final model
Tie-break	onset error compared at 1 ms resolution, ties go to the lower dev CER

Making alignment robust

- **Wildcard edges.** Audio is aligned in chunks of up to 60 s with a 1 s margin. A synthetic <star> symbol (best non-blank score minus 0.5) at both ends absorbs the extra audio instead of stretching the first or last word over it.
- **Chunk planning.** Chunks are cut at the longest pause near the end of each window, so no word is split.
- **Unpadded spans.** Word times come from the frames where letters are emitted, not padded into the surrounding silence (the padding would make subtitles appear about 0.5 s early).
- **Graceful fallback.** Words that cannot be aligned get times interpolated between their aligned neighbours.
- **Lag calibration.** A CTC model tends to emit a character slightly *after* the sound starts. On the dev clips our raw word starts are late by a near-constant amount, so that median lag is stored with the model (onset_shift = -47 ms) and applied to every aligned word.

Results

25 ms
mean |word-onset error| vs MFA (after lag calibration)

19 ms
median |onset error|

98.6%
word starts within 100 ms

7.8%
greedy character error rate (dev-clean)

55 ms
raw mean error before calibration (lag +47 ms)

epoch 11 of 11
best checkpoint

These dev figures use the lag measured on the same dev clips. The independent number is the test-set cue-onset error in section 10, measured on 27 files the aligner never saw.

Epoch	Train CTC loss	Dev CER	Raw onset ms	Calibrated onset ms	Within 100 ms
1	2.680	39.0%	66.1	27.0	97.7%
2	1.063	21.4%	56.8	26.4	98.1%
3	0.773	16.3%	56.4	26.9	97.5%

Epoch	Train CTC loss	Dev CER	Raw onset ms	Calibrated onset ms	Within 100 ms
4	0.647	13.4%	54.9	26.1	98.0%
5	0.567	12.0%	58.1	25.2	98.4%
6	0.506	10.5%	54.5	25.9	98.5%
7	0.460	9.7%	56.6	26.0	98.5%
8	0.421	9.3%	53.1	25.0	98.4%
9	0.362	8.0%	55.3	25.2	98.5%
10	0.338	8.0%	55.0	25.2	98.5%
11	0.329	7.8%	54.6	25.1	98.6%

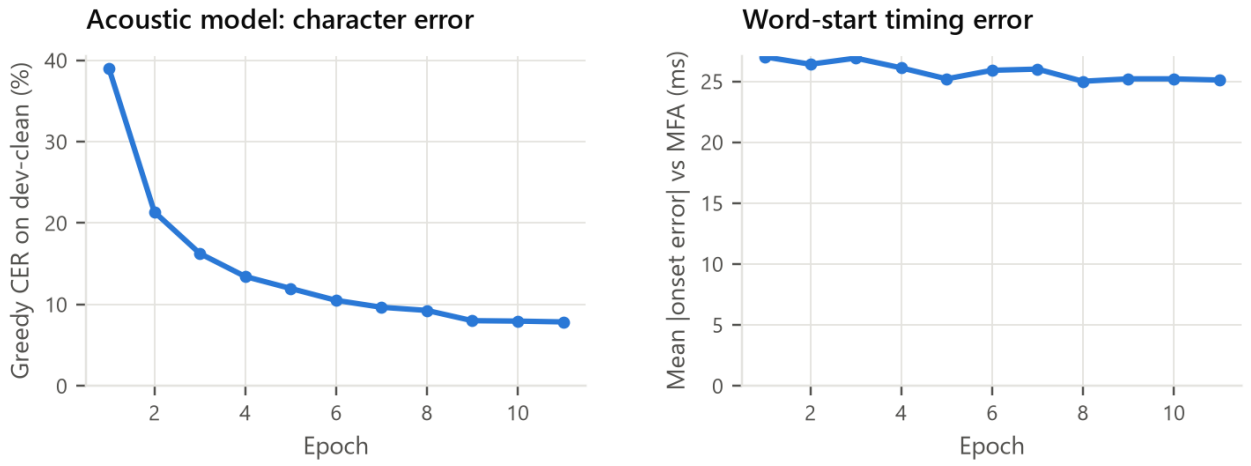


Figure 11. Left: recognition quality while training. Right: timing error, which is what the aligner is selected on.

Talking point

A 9.3M-parameter model trained from scratch on 100.6 hours times word starts on the dev clips with a mean error of 25 ms, with 98.6% within 100 ms. The MMS-300M aligner it replaces has 32 times more parameters and was pre-trained on hundreds of thousands of hours of speech; on the test files our aligner places cues more accurately (section 10).

INTEGRATION

9. Putting the models to work

The models do not act alone. In Mode A every disagreement between subtitle and audio becomes a candidate edit, scored by a weighted rule. The weights were re-tuned by grid search on the dev set after the language model changed, maximising F0.5 (precision counts twice as much as recall, because a wrong "correction" is worse than a missed one).

$$\text{score}(a \text{ over } s) = w_{conf} \cdot p_{ASR}(a) + w_{phon} \cdot \text{phon_sim}(s, a) + w_{LM} \cdot [\log P(a \mid ctx) - \log P(s \mid ctx)] + w_{oov} \cdot [s \text{ is not a dictionary word}] + w_{lex} \cdot [a \text{ is in the user's lexicon}]$$

The subtitle word s is replaced by the ASR word a when score $> \tau_{sub}$.

Tuned value	with distilgpt2	with our KN LM
w_conf (ASR confidence)	2	3
w_phon (sounds alike)	0.5	0.25
w_LM (language model)	0.1	0.2
w_oov (not a word)	0.25	0
tau_sub (threshold)	1.75	2.15
tau_ins (insertions)	0.95	0.95
tau_lm_ins (LM gate for insertions)	-3	-11.25
dev F0.5	0.7404	0.7473
dev true positives	858	906
dev false positives	286	305

With our language model the grid search moved weight towards ASR confidence, dropped the spelling flag and doubled the LM weight. The first test run showed that many dropped words were no longer put back: n-gram log-probabilities for an inserted word are much lower than distilgpt2's, so the LM gate for insertions sat outside the original search range. Widening that range on dev (tau_lm_ins -3 to -11.25) fixed it.

How the models cooperate (illustrative examples)

- **Fusion (LM):** "Hello their" vs heard "there": the LM prefers "Hello there", the phonetic similarity is high and Whisper is confident, so the word is replaced and logged with its evidence.
- **Punctuation:** adds the missing marks; in Mode A it only fills gaps and never removes the subtitler's marks.
- **Corrector:** repairs what is left, such as a duplicated word ("the the"), sentence by sentence, behind its guards.
- **Aligner:** times every final word, including the ones the corrector changed, which Whisper never timed.
- **POS tagger:** guides the dynamic-programming segmenter to split cues at natural phrase boundaries within 42 characters per line and 17 characters per second.

Tested and usable

- **95 Python test functions** (unit, property-based, integration and API) and **16 frontend tests**, including tests for each new model module: KN normalisation, BPE round-trips, beam search vs greedy, perceptron tagging, CTC alignment plumbing.
- **A web application** (FastAPI + React) wraps the pipeline: upload, live progress, a player with corrected captions and word-by-word highlighting from our aligner's times, a colour-coded diff with the evidence for each edit, an in-browser cue editor and downloads.

EVALUATION

10. End-to-end results: before and after

Same benchmark, same corruptions, same metrics: 8 LibriSpeech test-clean, 8 test-other and 11 TED-LIUM files with about 11% injected word errors plus timing and formatting faults.

Mode A: audio + corrupted subtitles

Metric	Pretrained models	Our models	Change
WER %	3.03	2.75	-0.28 (better)
CER %	1.78	1.55	-0.23 (better)
ERRANT F0.5	75.1	77.1	+2.0 (better)
Over-correction %	1.42	1.41	-0.01 (better)
Punctuation F1	72.6	70.1	-2.5 (worse)
Mean cue-onset error (ms)	50	39	-11 (better)
Onsets within 250 ms %	97.1	97.8	+0.7 (better)
Cues at <=17 CPS %	92.4	92.6	+0.2 (better)
Lines <=42 chars %	100.0	100.0	+0.0 (same)
Real-time factor	0.058	0.056	-0.002 (better)

Mode B: audio only

Metric	Pretrained models	Our models	Change
WER %	3.24	3.20	-0.04 (better)
CER %	1.93	1.88	-0.05 (better)
Punctuation F1	79.5	77.7	-1.8 (worse)
Mean cue-onset error (ms)	53	42	-11 (better)
Onsets within 250 ms %	96.8	97.3	+0.5 (better)
Cues at <=17 CPS %	91.7	92.1	+0.4 (better)
Lines <=42 chars %	100.0	100.0	+0.0 (same)
Real-time factor	0.063	0.053	-0.010 (better)

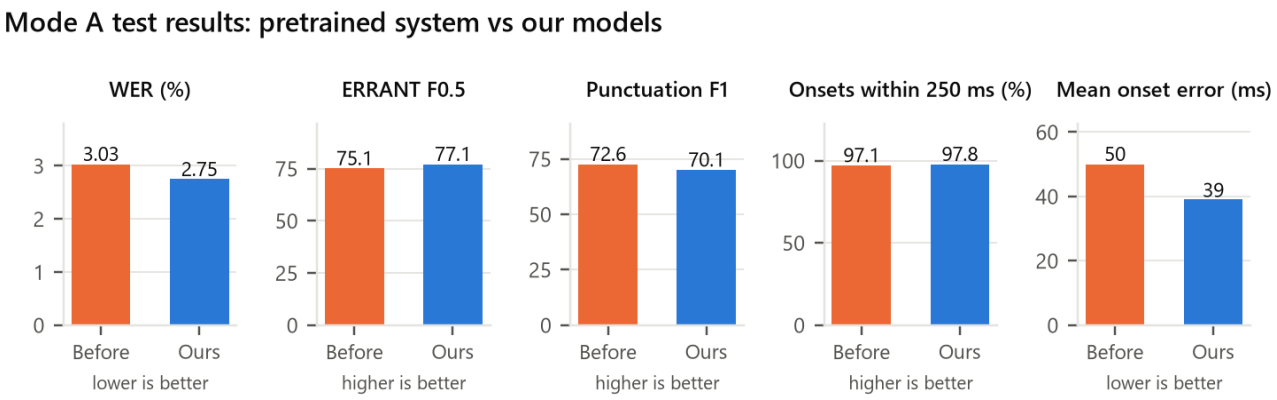


Figure 12. The headline metrics side by side (Mode A).

Timing: our aligner against the alternatives

Timing ablation on the 19 test files with MFA references (8 LibriSpeech test-clean clips and 11 TED talks). Only the word-timing step changes; the text is identical within each run.

Word-timing method	Cue onset error (ms)	Onsets within 100 ms	Onsets within 250 ms	Word |Δt| (ms)
Whisper's own timestamps (no aligner)	233	38.2%	63.4%	90
wav2vec2-base CTC, 95M (pretrained, before)	60	89.4%	96.9%	53
MMS-300M CTC, 300M (pretrained, before)	54	92.5%	96.6%	43
Our CTC aligner, 9.3M (from scratch)	**42**	**95.3%**	**97.4%**	**57**

Word |Δt| averages start and end errors. Measured separately, our word starts are better than MMS-300M's (about 33 ms vs 43 ms mean) but our word ends are worse (about 81 ms vs 42 ms). Ends only decide when a cue disappears, which the cue-timing rules then extend anyway.

Reading the results honestly

- **WER improved (3.03% to 2.75%), but mostly thanks to re-tuning.** Widening the search range of the fusion LM gate let more dropped words be put back (recall of dropped words rose from about 75% to 92%). The old system's own no-LM ablation suggests it would have gained similarly, so we do not credit this to the n-gram model.
- **Timing improved (50 to 39 ms mean cue-onset error; 97.1% to 97.8% within 250 ms).** This is a genuine win for our 9.3M-parameter aligner over the 300M-parameter MMS model.
- **Punctuation is the one place the pretrained model was better** (F1 72.6 vs 70.1 in Mode A, 79.5 vs 77.7 in Mode B). XLM-R-large was pre-trained on vastly more text than our 8.5M words.
- **The corrector does not help end to end** (WER 2.76% without it vs 2.80% with it on the ablation subset), exactly as with every pretrained grammar model in the first version: once fusion and the audio guard have run, little is left for it to fix. Its intrinsic result (13.9% to 4.3% WER on noisy sentences) shows the model itself works.
- **Speed is unchanged** (real-time factor 0.058 before, 0.056 now).

All systems on the new run (Mode A)

System	WER %	ERRANT F0.5	Onset ms
Corrupted input (no-op)	10.88	0.0	9365
B1: SymSpell + LanguageTool	8.82	43.3	9362
B2: ffsubsync (timing only)	11.45	0.0	5633
B3: raw Whisper replace	3.10	69.5	235
Ours (all self-trained models except Whisper)	2.75	77.1	39

ENGINEERING

11. Difficulties faced and how we solved them

Building models from scratch on a laptop over an unreliable connection is mostly an exercise in engineering under constraints. These are the problems that actually happened, in roughly the order we hit them.

Challenge	What went wrong	How we solved it
A 6.4 GB download over a slow, unstable network	A single connection to OpenSLR ran at about 90 KB/s (about 19 hours for train-clean-100); speed swung between 1 KB/s and 13 MB/s, and for a while a DNS outage made every mirror fail.	Wrote scripts/fetch_parallel.sh: 8 HTTP range requests spread over tested OpenSLR mirrors, each part resumable, with longer back-off and up to 3,000 retries.
Orphaned download processes corrupting files	Stopping the download task did not kill its background per-part loops; they kept starting curl, so after the restart two writers appended to the same parts.	Found and killed every leftover bash/curl process, then hardened the script: bytes are appended only on an HTTP 206 response, each run writes an owner token so stale loops stop, and a gzip -t check runs at the end.
A corrupted archive after the download finished	The archive failed its gzip integrity check. Streaming decompression located the first error exactly at the restart offset in part 3.	Instead of downloading 6.4 GB again, wrote a window patcher that compares a 4 KB sample of every 1 MB window with the server and re-fetches only the bad ones: 495 of 552 windows in part 3, about 520 MB, in 10 minutes. The repaired archive passed gzip -t and all 28,539 utterances (100.6 h) extracted.
Waiting on the repair	The aligner could not start training while the archive was broken.	Extracted the intact first 41% (stopping before the first bad byte) and ran a 2-epoch pilot on it to confirm that CTC trained from scratch converges and gives usable word onsets.
Interrupted working sessions	The AI agent session lost its connection during a DNS outage and later hit a usage limit, so work stopped twice mid-training.	Training ran as a background process with checkpoints every epoch (and every 500 steps for the aligner), so nothing was lost. Every neural trainer resumes from last.pt with its optimiser and scheduler state.
Only 6 GB of GPU memory	Two GPU jobs at once spill into shared memory and slow to a crawl.	Scheduled strictly one GPU job at a time: punctuation, then corrector, then aligner; batches sized by tokens or seconds of audio instead of sentence count; mixed precision.
Numerical overflow in half precision	The STFT power spectrum overflows float16; T5-style models overflow in fp16 as well.	The feature front end runs in fp32 and only the network uses fp16 autocast with a gradient scaler; the Transformer uses bfloat16, which has fp32's range.
16 GB of RAM, often only 2-6 GB free	8.5 million words as Python strings and dictionaries of n-grams would not fit.	n-grams packed into 64-bit integers and counted with NumPy; corpora streamed by generators; the punctuation dataset cached as compact integer arrays (one .npz file) rather than strings.

Challenge	What went wrong	How we solved it
Subtitles that may have no capital letters	A punctuation model that relies on capitals breaks on lower-case subtitles.	A separate casing embedding, plus 50% of training windows lower-cased at random, so the model works both ways (macro-F1 0.81 cased, 0.67 lower-cased).
A corrector that wants to rewrite everything	Sequence-to-sequence models tend to paraphrase correct text.	Trained with 33% clean copy pairs, plus an edit-ratio guard tuned to 15% and the audio-consistency guard; 97.6% of clean dev sentences pass through unchanged.
No training data for the corrector	There is no public corpus of faulty subtitles paired with corrections.	Generated pairs with a seeded noise model mixing synthetic typos and homophones with real whisper-tiny errors (the DARAG lesson: real errors beat purely synthetic ones), with fresh noise every epoch.
Unknown words in the language model	Without special handling, a typo scores the same as any rare word.	An open-vocabulary model: P(<unk>) times a character-bigram spelling probability.
Brown's 87 tags vs the 17 tags the segmenter needs	Compound tags (PPS+BEZ), title markers (-TL) and the have/do auxiliary ambiguity.	A mapping with lexical and look-ahead rules (have/do are AUX only before a verb).
Re-tuning after the swap	The cached dev features still contained distilgpt2 scores, so the old tuning was invalid.	Recomputed the features with our LM and re-ran the grid search; the text-stage cache is keyed on configuration, so the whole benchmark was re-run from scratch.
Forced alignment at chunk edges	Extra audio around a chunk was being forced onto the first or last word.	Wildcard <star> tokens at both ends of the CTC target absorb the margin.
CTC emits characters late	Raw word starts from our aligner were consistently late (median 46.7 ms on the dev clips).	Stored that median lag with the model as onset_shift (-46.7 ms) and applied it at inference; the selection metric is the error after calibration (25.1 ms mean).
Fusion stopped restoring dropped words	The first test run with our LM restored only about 65% of dropped words: the LM gate's search range had been sized for distilgpt2's scores.	Widened the grid for that gate on dev (dev F0.5 0.733 to 0.747); dropped-word recall rose to 92% and test WER fell from 3.26% to 2.75%.
Characters the corrector's tokenizer never saw	LibriSpeech text has no digits or exclamation marks, so the BPE alphabet lacks them; an untrained beam search also produced start tokens mid-sentence.	Sentences the BPE cannot represent exactly are passed through unchanged (BPE.covers); padding, start and unknown tokens are masked out during decoding.
A silently broken baseline	Baseline B2 (ffsubsync) had been failing since the project folder moved: its launcher still pointed at the old virtual environment.	The wrapper now runs ffsubsync through the current Python interpreter; B2 is back to its earlier numbers.
Choosing between near-identical checkpoints	Aligner epochs 8 and 11 differed by 0.1 ms of onset error but by 1.4 points of character error.	Onset error compared at 1 ms resolution with a CER tie-break, so epoch 11 (CER 7.85%) was selected.
A crash at interpreter exit	On Windows the aligner process ended with error 0xC0000409 after printing "done" (CUDA plus DataLoader workers).	Every output is saved before shutdown, so the crash costs nothing; noted rather than worked around.
Two workstreams in one folder	The models and the web application were built at the same time in the same uncommitted repository.	Strict file ownership between the two streams, stable public interfaces (pipeline.run, config keys), and a web app that hides options its configuration does not offer. While the configuration was mid-edit, the app showed a clear error instead of crashing.

REFLECTION

12. Lessons learned and future work

What we learned

- **Classical methods are not obsolete.** A 1998 smoothing method and a 2002 perceptron, implemented well, are competitive for narrow jobs and train in minutes.
- **Data design matters more than model size.** The corrector's quality comes from the noise model (real ASR confusions, clean copies), not from depth.
- **Evaluate the job, not the proxy.** The aligner is selected on word-onset error against MFA, not on recognition accuracy, because timing is what subtitles need.
- **Guard every learned component.** Edit-ratio and audio-consistency guards keep imperfect models safe.
- **Intrinsic gains do not always survive the pipeline.** The corrector cuts noisy-sentence WER by 69% on its own, yet changes almost nothing end to end, because fusion and the guards have already fixed what it would fix.
- **Re-tune after every swap.** A search range sized for one model silently capped the next one; widening it was worth half a point of WER.
- **Plan for failure.** Resumable checkpoints, resumable downloads and integrity checks turned outages into delays instead of lost work.

Future work

- Calibrate the aligner's word ends separately (or end each word where the next blank run starts); word ends are the one timing measure where MMS was better.
- Train the punctuation tagger on more, and more spoken, punctuated text (TED or OpenSubtitles transcripts) to close the 2.5-point gap to XLM-R.
- Train the acoustic model on all 960 hours of LibriSpeech, or replace the BiLSTM with a small Conformer.
- A neural LSTM language model, or KenLM-style pruning, to cut the LM file from 226 MB.
- Replace the rule-weighted fusion with a learned classifier (logistic regression on the same features).
- Evaluate on Indian-English recordings (our own clips) to measure accent robustness.
- Distil Whisper's knowledge into a smaller recogniser of our own, the one pretrained part left.

FOR THE VIVA

13. Presentation kit

Points worth saying

1. Five of the six learned components were designed, coded and trained by us; only Whisper is pretrained, and we can explain why it has to stay.
2. Our neural models total 30.0M parameters, against about 1.16 billion for the pretrained models they replace.
3. We implemented interpolated modified Kneser-Ney smoothing ourselves, with the Chen-Goodman discount estimates.
4. The LM stores 13.4 million n-grams as bit-packed 64-bit integers and trains in 114 seconds on a CPU.
5. An open-vocabulary trick (a character-bigram spelling model) lets the LM tell a typo from a rare word.
6. After re-tuning, fusion with our LM reaches 0.747 F0.5 on dev vs 0.740 with distilgpt2 (the small gain comes from a wider search range, not a better LM).
7. End to end, on the same 27 test files: WER 3.03% to 2.75%, ERRANT F0.5 75.1 to 77.1, cue-onset error 50 to 39 ms. We can also say honestly where we lost: punctuation F1, by 2.5 points.
8. Punctuation is framed as sequence labelling with a 2-layer BiLSTM over word, character-CNN and casing features.
9. Randomly lower-casing half of the training windows makes the punctuation model robust to caseless subtitles.
10. The punctuation tagger reaches macro-F1 0.81 with 46 times fewer parameters than XLM-R-large.
11. We wrote our own BPE tokenizer: 7,888 merges, 8,000-token vocabulary, exact round-trip decoding.
12. The corrector is a pre-norm Transformer encoder-decoder with tied embeddings and beam search with length normalisation.
13. With no parallel corpus available, we generated training pairs from a noise model that includes real whisper-tiny errors.
14. The corrector cuts noisy-dev WER from 13.9% to 4.3% and leaves 97.6% of clean sentences untouched.
15. Two guards (edit ratio and audio consistency) mean the audio always has the last word.
16. The POS tagger is an averaged perceptron: 97.2% accuracy with no neural network.
17. We mapped Brown's 87 tags to Universal Dependencies with look-ahead rules for auxiliaries.
18. The aligner computes its own log-mel features (our mel filterbank) and is trained with CTC loss and SpecAugment on 100.6 hours of speech.
19. CTC emits characters about 47 ms late; we measure that lag on dev and store it with the model as a calibration.
20. Forced alignment uses our NumPy Viterbi with wildcard edge tokens and pause-aware chunking.
21. Strict data hygiene: train for learning, dev for tuning, test once; LibriSpeech speakers are disjoint across splits.
22. We survived a corrupted 6.4 GB download, network stalls and an agent outage thanks to resumable checkpoints and a byte-range patcher.

Likely questions, with answers

Q. Why not simply use pretrained models?

For a course project the learning is in building them. Ours are 26-46 times smaller than the models they replace, run on a laptop, use only documented training data (no hidden test-set exposure) and carry no non-commercial licence restrictions.

Q. Then why keep Whisper?

Speech recognition at 3% WER needs hundreds of thousands of hours of training audio. With 100 hours we would get 15-30% WER, and the audio evidence the system relies on would be too unreliable. We use Whisper as a sensor and build everything that interprets its output.

Q. What is Kneser-Ney smoothing, and why not add-one?

Add-one gives too much probability to unseen events. Kneser-Ney subtracts a small discount from every seen count, passes that mass to lower orders, and uses continuation counts (how many contexts a word appears in) for those orders. It is the best classical n-gram smoother.

Q. How do you handle words the models have never seen?

The LM maps rare words to and multiplies by a character-level spelling probability; the punctuation tagger has a character CNN; the corrector splits unknown words into BPE pieces.

Q. Why a BiLSTM for punctuation rather than a Transformer?

With 8.5M training words and a 6 GB GPU, a BiLSTM is data-efficient and fast. The task needs only local context on both sides, which a bidirectional LSTM captures well.

Q. How did you train a corrector without labelled data?

We generated noisy/clean pairs from clean text with a seeded noise model: typos, homophones, dropped and doubled words, punctuation and case loss, and real whisper-tiny mistakes. A third of the pairs are clean copies.

Q. How do you stop the corrector from changing correct text?

Copy training, an edit-ratio guard (at most 15% of words changed) and the audio guard: a word Whisper heard confidently cannot be changed. 97.6% of clean sentences come out unchanged.

Q. What is CTC, and why does it need a blank symbol?

CTC trains a frame-level model from unaligned transcripts by summing over every way of spelling the transcript across the frames. The blank lets the model output nothing between characters and separates doubled letters, as in "ll".

Q. How is forced alignment different from recognition?

Recognition searches for the most likely text. Alignment already knows the text and searches only for the best timing of it, a constrained Viterbi path, so even a modest acoustic model can time words well.

Q. How do you know the timings are right?

We compare word starts with silver references from the Montreal Forced Aligner, report mean and median absolute onset error and the share within 100 and 250 ms, and pick the aligner checkpoint on that metric.

Q. How did you avoid data leakage?

Only train splits are used for learning. Dev splits select checkpoints and weights. The test benchmark is run once at the end, and LibriSpeech speakers do not overlap across splits.

Q. Is the perceptron really machine learning?

Yes. It is a linear classifier trained by error-driven updates over 127k sparse features, and averaging gives it the generalisation of a large-margin method. It is the model behind classic taggers such as NLTK's.

Q. Did removing the pretrained models hurt?

Not overall. On the same test set WER went from 3.03% to 2.75% and cue timing from 50 to 39 ms. Punctuation is 2.5 F1 points worse, and we say openly that most of the WER gain came from re-tuning a fusion threshold.

Q. Why does the corrector not lower WER end to end?

On its own it cuts noisy-sentence WER from 13.9% to 4.3%. In the pipeline, fusion with the audio has already fixed most errors and the audio guard blocks changes to confidently heard words, so little is left. Every pretrained grammar model behaved the same way in the first version.

Q. Where is your system still weaker?

Punctuation (XLM-R saw far more text) and word end times (our CTC aligner's ends are about 81 ms off vs 42 ms for MMS). Both have clear fixes: more punctuated training text and a separate end calibration.

Q. What was the hardest problem?

Getting 6.4 GB of audio over an unstable network and recovering from a corrupted archive without starting over. We solved it with parallel range downloads and a patcher that re-fetched only the damaged byte ranges.

Q. What would you do with more compute?

Train the aligner on 960 hours with a Conformer, replace rule-based fusion with a learned classifier, and eventually distil Whisper into a recogniser of our own.

ONE PAGE TO REMEMBER

14. Numbers cheat sheet

Model	Architecture	Size	Training data	Result	Training
Language model	Modified Kneser-Ney 4-gram	55,757 words; 13.4M n-grams	563,049 sentences / 8.54M words	ppl 284 (dev-clean)	114 s, CPU
Punctuation	Emb + char-CNN + 2x BiLSTM(256)	12,255,948 params	8.54M words	macro-F1 0.810	8 epochs, ~41 min
Corrector	Transformer 4+3 layers, d=256, 4 heads	8,409,344 params, BPE 8,000	513,355 sentences	WER 13.9% -> 4.3%	15 epochs, ~76 min
POS tagger	Averaged perceptron, 17 templates	127,047 features	54,473 Brown sentences	acc 97.21%	188 s, CPU
Aligner	Log-mel + 2 conv + 4x BiLSTM(320), CTC	9,317,533 params	100.6 h / 28,538 utts	onset 25 ms, CER 7.8%	11 epochs, 2 h 56 min

Code written

Component	Files	Lines
Kneser-Ney LM	3	397
Punctuation BiLSTM	2	408
Transformer corrector + BPE	4	639
Perceptron POS tagger	2	309
CTC acoustic aligner	3	704
Shared training corpus	1	56
Web backend (FastAPI)	8	1,231
Web frontend (React)	26	3,692

The whole `subfix` package is 4,972 lines of Python; 95 Python test functions and 16 frontend tests.

REFERENCE

15. Appendix

A. Reproducing every model

Command	Produces
`python experiments/train_lm.py`	Kneser-Ney 4-gram LM -> models/lm/kn4.npz
`python experiments/train_pos.py`	perceptron POS tagger -> models/pos/
`python experiments/train_punct.py --epochs 8`	BiLSTM punctuation tagger -> models/punct/
`python experiments/train_corrector.py --epochs 15`	BPE + Transformer corrector -> models/corrector/
`python experiments/train_aligner.py --epochs 25`	CTC acoustic model -> models/aligner/
`python experiments/tune_fusion.py`	re-tune fusion weights on dev -> configs/tuned.yaml
`scripts\run_all_experiments.ps1 -Fresh`	full benchmark -> results/
`python docs\report\build_report.py`	this report

Every trainer resumes from `last.pt` if it is interrupted; seeds are fixed.

B. Glossary

Term	Meaning
ASR	automatic speech recognition
BiLSTM	bidirectional long short-term memory network
BPE	byte-pair encoding, a subword tokenizer
CER / WER	character / word error rate
CTC	connectionist temporal classification, a loss for unaligned sequence labelling
CPS	characters per second, the subtitle reading speed
ERRANT	a toolkit that extracts and scores grammatical edits
F0.5	F-score weighting precision twice as much as recall
Forced alignment	finding when each word of a known transcript is spoken
KN	Kneser-Ney smoothing for n-gram language models
MFA	Montreal Forced Aligner, used for silver timing references
Perplexity	exp(average negative log-probability per word); lower is better
SpecAugment	masking bands of frequencies and time during training
UD / UPOS	Universal Dependencies part-of-speech tags

C. References

1. Chen, S. F. & Goodman, J. (1998). An empirical study of smoothing techniques for language modeling. Harvard TR-10-98.
2. Kneser, R. & Ney, H. (1995). Improved backing-off for M-gram language modeling. ICASSP.
3. Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. Neural Computation 9(8).
4. Vaswani, A. et al. (2017). Attention is all you need. NeurIPS.

5. Sennrich, R., Haddow, B. & Birch, A. (2016). Neural machine translation of rare words with subword units. ACL.
6. Wu, Y. et al. (2016). Google's neural machine translation system (length normalisation). arXiv:1609.08144.
7. Collins, M. (2002). Discriminative training methods for hidden Markov models: perceptron algorithms. EMNLP.
8. Graves, A. et al. (2006). Connectionist temporal classification. ICML.
9. Park, D. S. et al. (2019). SpecAugment. Interspeech.
10. Loshchilov, I. & Hutter, F. (2019). Decoupled weight decay regularization (AdamW). ICLR.
11. Panayotov, V. et al. (2015). LibriSpeech: an ASR corpus based on public domain audio books. ICASSP.
12. Meister, A. et al. (2023). LibriSpeech-PC: punctuation and capitalization restoration benchmark. arXiv:2310.02943.
13. Francis, W. N. & Kucera, H. (1979). Brown Corpus Manual. Brown University.
14. Petrov, S., Das, D. & McDonald, R. (2012). A universal part-of-speech tagset. LREC.
15. Radford, A. et al. (2023). Robust speech recognition via large-scale weak supervision (Whisper). ICML.
16. McAuliffe, M. et al. (2017). Montreal Forced Aligner. Interspeech.
17. Bryant, C., Felice, M. & Briscoe, T. (2017). Automatic annotation and evaluation of error types for GEC (ERRANT). ACL.